

PRACTICAL WEEK 8

Task 1:

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    /* malloc() */
    int *a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
    {
        a[i] = i + 1;
    }

    printf("Values using malloc:\n");

    for (i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n");

    /* calloc() */
    int *b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
    }
}
```

```

        free(a);
        return 1;
    }

    printf("\nValues using calloc:\n");

    for (i = 0; i < 5; i++)
    {
        printf("%d ", b[i]);
    }

    printf("\n");

    /* realloc() */
    a = realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
        printf("realloc failed\n");
        free(b);
        return 1;
    }

    for (i = 5; i < 10; i++)
    {
        a[i] = i + 1;
    }

    printf("\nValues after realloc:\n");

    for (i = 0; i < 10; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n");

    /* free() */
    free(a);
    free(b);

    printf("\nMemory successfully freed.\n");

    return 0;
}

```

Task 2:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int global_data = 100;

int main()
{
    int local_data = 200;

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());

    printf("global_data address: %p\n", (void *)&global_data);
    printf("local_data address : %p\n", (void *)&local_data);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process

        printf("\nChild process:\n");
        printf("Child PID: %d\n", getpid());

        printf("\nBefore modification:\n");
        printf("global_data = %d\n", global_data);
        printf("local_data = %d\n", local_data);

        // Modify the data
        global_data = 500;
        local_data = 600;

        printf("\nAfter child modification:\n");
        printf("global_data = %d\n", global_data);
        printf("local_data = %d\n", local_data);
    }
}
```

```

    printf("\nChild addresses:\n");
    printf("global_data address: %p\n", (void *)&global_data);
    printf("local_data address : %p\n", (void *)&local_data);

    exit(0);
}
else
{
    // Parent waits for child

    wait(NULL);

    printf("\nParent process after child modification:\n");

    printf("global_data = %d\n", global_data);
    printf("local_data = %d\n", local_data);

    printf("\nParent addresses:\n");
    printf("global_data address: %p\n", (void *)&global_data);
    printf("local_data address : %p\n", (void *)&local_data);
}

return 0;
}

```

Output:

```
zsh: command not found: valgrind
rishivarma@rishis-MacBook-Air Exp 8 practical % ./memory_allocation
Values using malloc:
1 2 3 4 5

Values using calloc:
0 0 0 0 0

Values after realloc:
1 2 3 4 5 6 7 8 9 10

Memory successfully freed.
rishivarma@rishis-MacBook-Air Exp 8 practical % nano copy_on_write.c
rishivarma@rishis-MacBook-Air Exp 8 practical % gcc copy_on_write.c -o copy_on_write
rishivarma@rishis-MacBook-Air Exp 8 practical % ./copy_on_write
Before fork:
Parent PID: 34589
global_data address: 0x1005b8000
local_data address : 0x16f84efa8

Child process:
Child PID: 34591

Before modification:
global_data = 100
local_data = 200

After child modification:
global_data = 500
local_data = 600

Child addresses:
global_data address: 0x1005b8000
local_data address : 0x16f84efa8

Parent process after child modification:
global_data = 100
local_data = 200

Parent addresses:
global_data address: 0x1005b8000
local_data address : 0x16f84efa8
rishivarma@rishis-MacBook-Air Exp 8 practical %
```